Video Presentation Script & Guide

Experiment 3: Sampling, Aliasing and Whittaker-Shannon Sinc Reconstruction

Subject: Software-Based Digital Communication Laboratory (ECE Dept.)

Presenter: Apurba Maity (Roll: `34900324001`)

Institution: Cooch Behar Govt. Engineering College

⏱️ **Duration: 4:30 – 5:00 min** 🎯 **Core Focus: Python Code & Plot Walkthrough**

🌟 **Theory vs Practicality Deep-Dive**

📋 Pre-Recording Setup & Windows to Open

🖥️ **Window 1 (Main Focus — 75% of video):** VS Code open with `experiment_03.py` and `Experiment_03_Lab_Report.ipynb`.

📊 **Window 2 (Plots Viewer):** High-resolution figures open from the `plots/` directory.

🌐 **Window 3 (Closure — 30 seconds):** Interactive Simulation Studio (`index.html`) in full-screen browser.

🐙 **Window 4 (Closing):** Public GitHub Repository (`SecretiveCodeRunner/Digital_Communication_Lab`).

Scene 1: Introduction & Objective

0:00 – 0:40

🖥️ **ON-SCREEN ACTION**

Show VS Code editor open to `Experiment_03_Lab_Report.ipynb` on the title banner, displaying your Name, Roll Number, Department, and Subject header.

🎙️ **SPOKEN NARRATION (READ ALOUD STEADILY)**

"Hello everyone and respected faculty. My name is **Apurba Maity**, Roll Number **34900324001**, from the Department of Electronics and Communication Engineering at Cooch Behar Government Engineering College.

Today, I will be presenting **Experiment 3** of our Software-Based Digital Communication Laboratory: **Sampling, Aliasing, and Whittaker-Shannon Sinc Reconstruction**.

Our core objective today is to explore the **Nyquist-Shannon Sampling Theorem** through Python numerical simulation, implement **ideal sinc interpolation** from first principles, observe **spectral aliasing** when undersampled, and most importantly, examine the critical differences between **pure theoretical mathematical assumptions and practical computational realities**."

Scene 2: Theoretical Formulation & Signal Model

0:40 – 1:25

🖥️ **ON-SCREEN ACTION**

Scroll to Section 1 of the Notebook, highlighting the mathematical formulas for continuous test signal $x(t)$, Nyquist rate f_{Nyquist}, and the aliasing folding equation.

$$x(t) = \sin(2\pi \cdot 100t) + 0.5\sin(2\pi \cdot 300t)$$

$$f_s \geq 2f_{\max} = 2 \times 300 \text{ Hz} = 600 \text{ Hz}$$

$$\hat{x}(t) = \sum_{n=-\infty}^{\infty} x[n] \cdot \text{sinc}\left(f_s(t - nT_s)\right)$$

$$f_{\text{alias}} = |f_2 - k \cdot f_s| = |300 - 1 \times 400| = 100 \text{ Hz}$$

🎙️ **SPOKEN NARRATION**

"Let us first establish our theoretical framework. We construct a deterministic continuous two-tone signal: **x of t equals sine of 2 pi times 100 t, plus 0.5 sine of 2 pi times 300 t**.

Here, the fundamental frequency is **100 Hertz** and the highest harmonic is **300 Hertz**. Thus, our maximum signal bandwidth is **300 Hertz**.

According to the **Nyquist-Shannon Sampling Theorem**, exact lossless recovery requires a sampling frequency of at least twice the maximum frequency, which is **2 times 300 = 600 Hertz**.

Continuous-time reconstruction is governed by the **Whittaker-Shannon sinc interpolation formula**, which convolves discrete samples with ideal sinc pulses.

If we violate this by undersampling at **400 Hertz**, the high-frequency 300 Hertz component folds across the 200 Hertz Nyquist boundary down to **100 Hertz**, according to the formula: **f-alias equals the absolute value of 300 minus 400, which equals 100 Hertz**.

Now, let us examine how we implemented this in Python without using external black-box toolboxes."

Scene 3: Python Code Architecture & DSP Walkthrough

1:25 – 2:15

🖥️ **ON-SCREEN ACTION**

Switch to `experiment_03.py` in VS Code. Highlight the `sinc_reconstruction()` function (lines 53–63) and `calculate_spectrum()` (lines 65–73).

```
def sinc_reconstruction(sample_times, sample_values, t_fine, fs):
    # Vectorized 2D broadcast: shape (N_samples, M_eval_points)
    dt_matrix = t_fine[None, :] - sample_times[:, None]
    sinc_matrix = np.sinc(fs * dt_matrix)  # np.sinc computes sin(pi*x)/(pi*x)
    # Sum overlapping sinc pulses across all discrete samples
    x_hat = np.dot(sample_values, sinc_matrix)
    return x_hat
```

🎙️ **SPOKEN NARRATION**

"Here in our Python implementation, we avoided black-box DSP functions and implemented the core mathematics directly using NumPy.

As shown in lines 53 to 63 in `sinc_reconstruction`:

- To evaluate the Whittaker-Shannon summation efficiently for thousands of time points, we construct a 2D distance matrix using array broadcasting: `dt_matrix = t_fine[None, :] - sample_times[:, None]`.
- Passing this into `np.sinc(fs * dt_matrix)` creates our full interpolation kernel grid.
- A single vectorized dot product `np.dot(sample_values, sinc_matrix)` instantly sums all overlapping sinc pulses across the entire continuous evaluation grid in a fraction of a millisecond.

For frequency analysis, lines 65 to 73 use `np.fft.rfft` to compute the single-sided discrete Fourier spectrum, normalized by **2 divided by N** to obtain exact physical peak amplitudes.

Furthermore, in lines 110 to 125, we implemented sample boundary padding of **20 extra samples** on either end, eliminating artificial edge-truncation errors in our numerical reconstruction."

Scene 4: Plot Walkthrough & Empirical Validation

2:15 – 3:25

🖥️ **ON-SCREEN ACTION**

Display the 4 generated Matplotlib figures from the `plots/` folder or scroll through Section 3 of the Jupyter Notebook.

🎙️ **SPOKEN NARRATION**

"Let us now examine our empirical results across the 4 generated figures:

1. Discrete Sample Stems (Figure 1): Over our 50 millisecond test duration:

- At **1800 Hertz** (top), 91 dense samples capture 6 samples per cycle of the 300 Hertz wave.
- At **600 Hertz** (middle), 31 samples capture exactly 2 samples per cycle.
- At **400 Hertz** (bottom), only 21 samples are recorded—barely 1.3 samples per cycle—completely missing the sinusoidal peaks.

2. Reconstructed Waveforms & FFT Spectra (Figures 2 & 3):

- In the **1800 Hertz Oversampled case**, the reconstructed dashed curve overlays the original black reference signal perfectly, yielding a near-zero Mean Squared Error of **5.0 times 10 to the minus 6**. The FFT spectrum cleanly recovers both the 100 Hertz and 300 Hertz peaks.
- In the **400 Hertz Undersampled case**, the reconstructed waveform is heavily distorted. Looking at the FFT spectrum, the 300 Hertz peak has completely vanished from its true position and folded directly onto **100 Hertz**, exactly confirming our theoretical derivation: **absolute value of 300 minus 400 equals 100 Hertz**!

3. Time-Domain Error (Figure 4): The error waveform confirms this quantitatively: oversampling yields a flat zero baseline, while undersampling results in a massive oscillating error with a Mean Squared Error of **0.25**."

Scene 5: 🌟 Theory vs. Practicality & Computational Realities

3:25 – 4:15

🖥️ **ON-SCREEN ACTION**

Scroll to **Section 4: Theory vs. Practicality** in the Notebook. Point with mouse cursor to the comparative table and error numbers.

💡 **CRITICAL VIVA & DEFENSE INSIGHT**

Explain why $f_s = 600\ \text{Hz}$ produced $\text{MSE} \approx 0.1235$ despite meeting the Nyquist rate, and explain the physical necessity of Anti-Aliasing filters and finite-window padding.

🎙️ **SPOKEN NARRATION**

"Now, let us address the most insightful question of this experiment: **Where does pure mathematical theory diverge from practical simulation and physical hardware?**

1. Phase Nulling at Critical Nyquist (600 Hertz):
In textbooks, theory states that **f_s = 2 f_max** guarantees exact signal reconstruction. However, in our simulation, the 600 Hertz critical case produced an MSE of **0.1235**! Why did this happen?

Our test signal is a pure sine wave: **sine of 2 pi times 300 t**. When sampled at intervals of **1 divided by 600 seconds**, every single sample falls exactly on a zero crossing: **sine of n pi is identically zero**!

Every discrete sample of the 300 Hertz tone evaluated to zero! The sinc interpolator was therefore blind to the 300 Hertz wave and only reconstructed the 100 Hertz tone. The missing harmonic power is exactly **0.5 squared divided by 2 = 0.125**, which matches our measured MSE of **0.1235**!

This proves that in real-world DSP and ADC design, we cannot operate right at the theoretical boundary; we must strictly oversample (for example at **2.2 times f_max**) with a safety guard band to avoid phase nulling.

2. Infinite Sinc Summation vs. Finite Observation Window:
Theoretical Whittaker-Shannon interpolation requires an infinite summation from minus infinity to plus infinity. Real DSP processors and digital scopes operate on finite time windows (such as 50 milliseconds). Truncating the sinc series introduces boundary ripples, which we resolved by implementing sample padding of 20 extra points.

3. Physical Anti-Aliasing Filtering:
In software math, we can inspect and predict folded frequencies. But in physical ADC

hardware, once an out-of-band high-frequency signal aliases into baseband, it is mathematically inseparable from legitimate data. Therefore, an analog active Low-Pass Anti-Aliasing Filter must always precede the hardware ADC."

Scene 6: Interactive Web Simulator (Bonus Closure)

4:15 – 4:45

🖥️ **ON-SCREEN ACTION**

Switch to browser window displaying `index.html`. Click preset buttons (**1800 Hz**, **600 Hz**, **400 Hz**) and briefly drag the f_s slider.

🎙️ **SPOKEN NARRATION**

"As a bonus interactive closure, we also compiled this lightweight 60 FPS HTML5 simulation studio.

Here, we can dynamically drag the sampling frequency slider from 1800 Hertz down to 300 Hertz. Notice how the reconstructed green waveform morphs in real time, and in the live FFT spectrum below, you can watch the aliased red peak sweep continuously across the folding boundary as sampling frequency decreases. We can also synthesize Web Audio tones to hear how aliasing lowers the perceived auditory pitch."

Scene 7: Summary & Conclusion

4:45 – 5:00

🖥️ **ON-SCREEN ACTION**

Switch to GitHub repository page (`SecretiveCodeRunner/Digital_Communication_Lab`).

🎙️ **SPOKEN NARRATION**

"In conclusion, this experiment successfully verified the Nyquist-Shannon Sampling Theorem, implemented vectorized sinc reconstruction in Python, validated spectral aliasing folding, and clarified the boundary between continuous theory and discrete DSP implementation.

The complete Python code, Jupyter lab notebook, printable study guides, and the interactive studio are published on my open-source GitHub repository.

Thank you very much for your time and attention!"

📊 Summary Results Table & Viva Q&A Defense

Regime	Sampling Rate (f_s)	Ratio	Samples	MSE (V^2)	RMSE (V)	Harmonic Status
Oversampled	1800 Hz	3.0x Nyquist	91	5.0×10^{-6}	0.0022	Clean recovery at 300 Hz
Critical	600 Hz	1.0x Nyquist	31	0.1235	0.3514	Phase Null ($\sin(n\pi) \equiv 0$)
Undersampled	400 Hz	0.67x Nyquist	21	0.2500	0.5000	**Aliased to 100 Hz**

🎯 Predictive Viva Q&A Defense Sheet

Q1: Why did Critical Sampling ($f_s = 600\ \mathrm{Hz}$) yield an MSE of 0.1235 instead of 0?
Answer: The 300 Hz harmonic was $0.5\sin(2\pi \cdot 300t)$. When sampled at intervals of $T_s = 1/600\ \mathrm{s}$, the sample times are $t_n = n/600$, resulting in $0.5\sin(n\pi) \equiv 0$ for all integers n. Thus, all discrete samples of the 300 Hz component were identically zero! Sinc interpolation could only recover the 100 Hz tone. The energy of the lost 300 Hz tone is $\frac{1}{2}A_2^2 = \frac{1}{2}(0.5)^2 = 0.125\ \mathrm{V}^2$, which matches our measured MSE of $0.1235\ \mathrm{V}^2$.

Q2: How does your Python code implement sinc reconstruction without slow loops?
Answer: Through 2D array broadcasting and matrix multiplication: `dt = t_fine[None, :] - sample_times[:, None]`, evaluated with `np.sinc(fs * dt)` and summed via `np.dot(sample_values, sinc_matrix)`.

Q3: Why is an Anti-Aliasing Filter essential in hardware ADCs?
Answer: Once an out-of-band analog frequency higher than $f_s/2$ enters the ADC sampler, it aliases into baseband $[0, f_s/2]$ and produces identical discrete samples as a legitimate low-frequency signal. It is mathematically impossible to distinguish or filter out aliased components after digitization. Therefore, an analog active Low-Pass Filter must attenuate out-of-band frequencies *before* the ADC.

Q4: Why did we pad samples ($N_{\text{pad}} = 20$) in the time domain?
Answer: Theoretical Whittaker-Shannon interpolation requires an infinite summation ($-\infty \le n \le +\infty$). In numerical simulation over a finite window $[0, T]$, truncating the sinc series leaves points near $t = 0$ and $t = T$ without neighboring sinc pulses, causing boundary edge

distortion. Evaluating samples slightly beyond the plotting window ensures full sinc overlap across the entire evaluation interval.